

Tarea 2 – OpenCL y CUDA

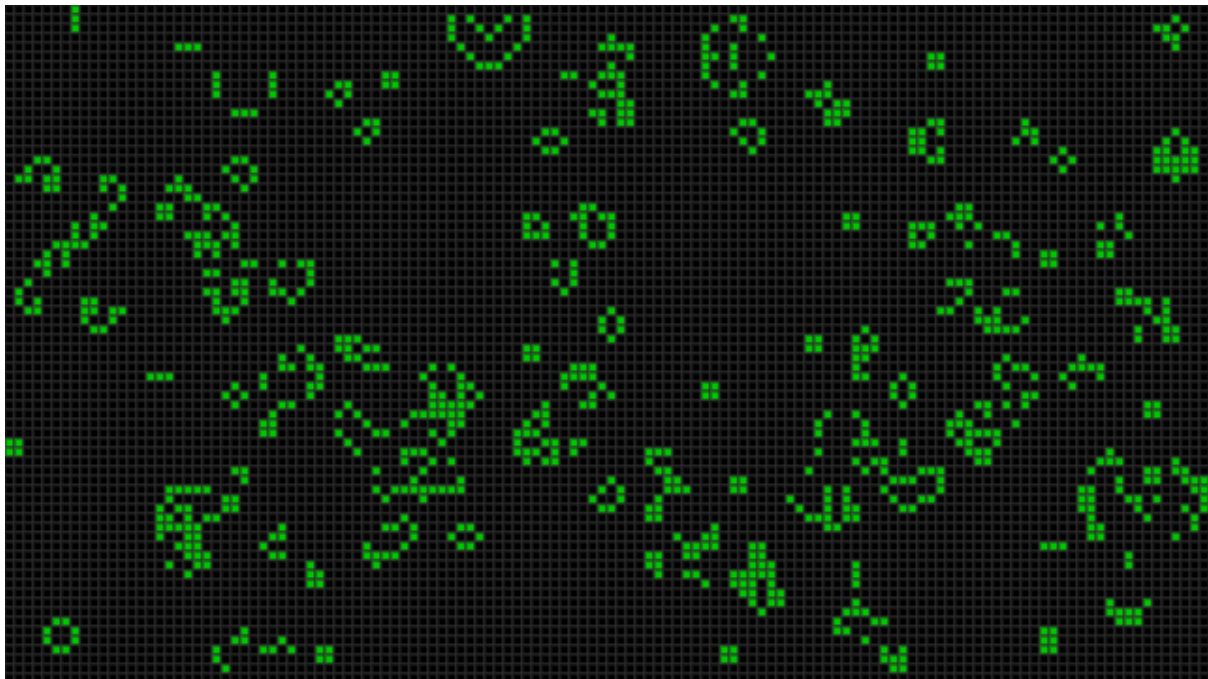
Profesora: Nancy Hitschfeld K.
Auxiliar: Diego García y Vicente González

1. Introducción

Con esta tarea se busca aprender como implementar algoritmos en GPU y optimizarlos. Para ello deben elegir un problema paralelizable a resolver, implementarlo en CPU, Cuda y OpenCL, todo en C++. Se debe elegir resolver uno de los problemas, el juego de la vida o el n –body problem.

2. Problemas a resolver

2.1. El juego de la Vida de Conway



El Juego de la Vida de Conway es un popular autómatas celular creado por el matemático británico John Horton Conway en 1970. Se trata de un modelo matemático que simula el crecimiento y evolución de células en una cuadrícula bidimensional. Cada célula puede estar viva o muerta y su estado evoluciona según una serie de reglas predefinidas. El objetivo de esta tarea desarrollar una implementación en GPU del Juego de la Vida de Conway utilizando CUDA y OpenCL, y probar diferentes técnicas de optimización para mejorar el rendimiento de la implementación.

Las reglas que debe seguir el juego de la vida son las siguientes:

- Una célula nace de un espacio muerto si tiene 3 vecinos vivos.
- Una célula sobrevive en la próxima generación si tiene 2 o 3 vecinos.

Debe realizar una implementación serial en CPU e implementar el algoritmo tanto en OpenCL como en CUDA. Debe medir el número de células evaluadas por segundo para varios tamaños de grilla $N \times M$. Esta tarea se puede desarrollar también en parejas, en donde un integrante la desarrolla en CUDA y el otro en OpenCL (también pueden desarrollar todo ambos).

Además, debe elegir al menos dos de las siguientes opciones y medir su desempeño. Los resultados deben ser expuestos en un reporte que corresponde al 50% de la nota final:

1. Usando ifs para preguntar si la vecindad está viva.
2. Usando tamaños de bloque tanto múltiplos de 32 como no múltiplos de 32.
3. Usando arreglos de dos dimensiones en vez de un mapeo a arreglo de una dimensión.
4. Usando memoria local por bloque y sin memoria local.

En el siguiente enlace hay una implementación completa con ejemplos desde el cual puede basar su implementación: <http://www.marekfiser.com/Projects/Conways-Game-of-Life-on-GPU-using-CUDA>.

2.2. N –body problem



El problema n –body se refiere a la simulación de la interacción gravitatoria entre un gran número de cuerpos, como estrellas, planetas o galaxias. La solución a este problema implica el cálculo de la fuerza gravitatoria entre cada par de cuerpos, lo que lleva a un algoritmo de complejidad $O(n^2)$. Sin embargo, esta complejidad hace que el algoritmo sea impracticable para un gran número de cuerpos. Una solución a este problema es utilizar técnicas de computación paralela, como CUDA, para acelerar el cálculo de la fuerza gravitatoria y obtener una simulación en tiempo real.

Debe realizar una implementación serial en CPU e implementar el algoritmo tanto en OpenCL como en CUDA. Debe medir el número de células evaluadas por segundo para varios tamaños de grilla $N \times M$. Esta tarea se puede desarrollar también en parejas, en donde un

integrante la desarrolla en CUDA y el otro en OpenCL (también pueden desarrollar todo ambos).

Además, implementar las siguientes opciones para medir su impacto en el desempeño de la simulación. Los resultados de estas comparaciones se deben presentar en un informe que represente el 50% de la nota final:

1. Usando tamaños de bloque tanto múltiplos de 32 como no múltiplos de 32.
2. Usando arreglos de dos dimensiones en vez de un mapeo a arreglo de una dimensión.
3. Usando memoria local por bloque y sin memoria local.

En el siguiente enlace hay una implementación completa con ejemplos desde el cual puede basar su implementación: <https://www.evl.uic.edu/sjames/cs525/project2.html>.

3. Contenido del informe

A continuación se detalla una guía del contenido mínimo que debe tener el informe. El orden o tipo de sección/subsección lo pueden decidir ustedes.

1. Introducción (**0.5pt**)
 - Breve descripción del problema.
 - Resultados esperados.
2. Implementación (**1.0pt**)
 - Describir implementación secuencial.
 - Describir implementación paralela en CUDA.
 - Describir implementación paralela en OpenCL.
 - Describir variaciones en configuración.
3. Resultados (**2.0pt**)
 - Resultados por tamaño de grilla (CPU/GPU).
 - Resultados de experimentos para opciones elegidas.
4. Análisis de resultados (**1.5pt**)
 - *Speed-up* comparando versiones paralelas con el resultado en CPU.
 - Estudio tamaño de grillas.
 - Calcular desde qué tamaño de grilla es más conveniente usar la GPU.
5. Conclusiones (**1.0pt**)

4. Información importante

- Se debe incluir README sobre cómo ejecutar su programa. Si no se incluye un README su tarea será evaluada con la nota mínima.
- La fecha de entrega es aquella que salga en U-Cursos. Si tiene algún problema de cualquier índole por la situación actual nos pueden comunicar por correo.
- Se recomienda hacer los experimentos en Pandas y Jupyter, de Python, con Visual studio Code, ya que así se pueden desarrollar de manera más simple la limpieza de datos, la creación de los gráficos con matplotlib, y el análisis de los resultados.